

HACKATHON THEME

Intelligent Systems

Research • Engineer • Build • Impact

Build software that thinks, adapts, and creates measurable impact.

The **Intelligent Systems** theme challenges participants to design and develop innovative software solutions that solve complex real-world problems through computational thinking, intelligent decision-making, research, and modern software engineering.

This document defines the broad direction of the hackathon. The detailed problem statements, constraints, and participant-facing challenge briefs will be released separately approximately **48 hours before the hackathon**.

Application Areas

Healthcare & Well-being | Smart Cities & Urban Infrastructure | Sustainability & Environment
Cybersecurity & Digital Trust | Enterprise Productivity & Intelligent Automation

Theme Overview

In today's digital world, software is expected to do more than automate tasks. Intelligent systems must be capable of analyzing information, adapting to changing conditions, optimizing decisions, enhancing human capabilities, and creating measurable impact.

The **Intelligent Systems** theme encourages participants to combine research, computational thinking, modern software engineering, and creative problem-solving to build systems that are reliable, secure, measurable, and capable of addressing meaningful real-world challenges.

The theme is intentionally broad. Teams may explore different application areas and technology stacks. The detailed problems released at T-48 hours will narrow that broad direction into specific research-grade challenges.

Important scope principle: the hackathon is not asking teams to reproduce a complete production system in a weekend. Teams are expected to build a **prototype-scale proof of the hardest technical idea** and support it with clear reasoning and evidence.

Theme Objectives

Participants should aim to build software systems that:

- Solve meaningful real-world challenges through technology.
- Demonstrate intelligent decision-making or computational capabilities.
- Apply strong software engineering principles and system design.
- Utilize data effectively to improve performance and user experience.
- Be designed with scalability, security, maintainability, and real-world deployment in mind.
- Deliver measurable impact for individuals, organizations, or society.
- Showcase innovation through thoughtful engineering, research, and evidence rather than complexity alone.

Core Principles

Every solution developed under this theme should emphasize the following principles.

Intelligent Problem Solving

Design systems that can analyze information, automate processes, optimize outcomes, or assist human decision-making. Intelligence may come from algorithms, distributed computation, formal reasoning, optimization, data processing, or machine learning where appropriate.

Engineering Excellence

Focus on clean architecture, maintainable code, modular design, observability, testing, performance optimization, fault tolerance, and software quality.

Innovation

Introduce original ideas, creative workflows, novel architectures, new protocols, or thoughtful applications of existing technologies to solve problems more effectively.

Practical Impact

Address genuine user or organizational needs and design solutions that can realistically be adopted beyond the hackathon environment.

Research & Evidence

Treat the challenge as an engineering research problem. Define assumptions, investigate relevant approaches, identify trade-offs, and validate important claims with measurable evidence.

Scalability & Reliability

Build systems capable of handling growth, recovering from failures, adapting to changing requirements, and maintaining consistent performance.

Technology Scope

Participants are free to use suitable programming languages, frameworks, cloud platforms, databases, APIs, and development tools.

Technology selection should be driven by the problem being solved rather than by the technology itself.

The technologies below are **illustrative rather than prescriptive**. A team does not gain additional credit simply for using a more complex stack.

Technology Scope

Projects may incorporate technologies such as:

- Artificial Intelligence & Machine Learning
- Cloud Computing
- Data Analytics
- Distributed Systems
- Cybersecurity
- APIs & Microservices
- Real-Time Data Processing
- Computer Vision
- Natural Language Processing
- Blockchain
- Mobile & Web Technologies
- Advanced Algorithms
- Graph Computing
- Privacy-Preserving Computation
- Formal Methods
- Simulation Systems

Technology Selection Principle

Teams are not expected to use every listed technology. The strongest solutions will choose technology because it is necessary to solve the underlying problem, not merely because it is technically impressive.

Application Areas

01 Healthcare & Well-being

Design intelligent software solutions that improve healthcare accessibility, patient care, diagnostics, emergency response, or health management.

02 Smart Cities & Urban Infrastructure

Build systems that enhance transportation, public services, utilities, resource management, and urban efficiency.

03 Sustainability & Environment

Create technology-driven solutions that promote environmental sustainability, efficient resource utilization, climate resilience, and sustainable development.

04 Cybersecurity & Digital Trust

Develop secure, intelligent systems that strengthen digital security, privacy, identity management, and protection against cyber threats.

05 Enterprise Productivity & Intelligent Automation

Engineer software that improves organizational productivity, automates workflows, enhances collaboration, and supports intelligent business decision-making.

Challenge Expectations

The hackathon is designed to encourage teams to move beyond conventional application development.

A strong submission should demonstrate:

1. **A clearly defined real-world problem** with measurable impact.
2. **A thoughtful system architecture** rather than a collection of disconnected features.
3. **A meaningful computational component** such as optimization, reasoning, real-time processing, distributed coordination, advanced analytics, security, or AI where appropriate.
4. **Evidence of engineering quality** through testing, observability, fault handling, performance evaluation, and maintainability.
5. **Scalability and reliability considerations** appropriate to the problem.
6. **Security and privacy by design** where sensitive data or trust relationships are involved.
7. **A research question or technical claim** that gives the team something meaningful to test.
8. **A measurable demonstration** showing that the proposed system works and produces meaningful results.
9. **A validation approach** appropriate to the chosen problem, such as a baseline, controlled experiment, benchmark, simulation, or reproducible test.

Theme Philosophy

The goal is not simply to develop software that works.

Engineer intelligent systems that solve problems, improve lives, and contribute to the future of technology.

Participants are encouraged to approach every challenge with curiosity, design robust systems with real-world applicability, and build solutions that are not only functional but also scalable, secure, and meaningful.

Preparing for the Challenge

This theme document is intended to help participants prepare their thinking before the detailed problem statements are released. It is an opportunity to prepare your **thinking**.

1. **Understand the theme.** Identify the domains and technical patterns that interest your team.
2. **Build research readiness.** Review foundational concepts related to distributed systems, intelligent decision-making, security, optimization, data systems, simulation, or other relevant areas.
3. **Prepare reusable tooling.** You may prepare development environments, boilerplate, testing utilities, visualization components, or general-purpose libraries, subject to the event rules.
4. **Do not pre-build a hidden solution.** The detailed problem is released at T-48 hours; teams should not attempt to reverse-engineer or obtain unreleased problem statements.
5. **After T-48, narrow aggressively.** Choose the problem whose hardest technical constraint matches your team's strengths, then define the smallest credible prototype that can prove the idea.

The intended preparation mindset

Understand the space → **Choose the right challenge** → **Prove the hardest idea**

Research. Engineer. Build. Impact.

From intelligent algorithms to distributed systems,
from secure infrastructure to real-time computation,
build the software systems that shape what comes next.

INTELLIGENT SYSTEMS

Research • Engineer • Build • Impact